C-to-RTL Equivalence Checking for High-level Synthesis

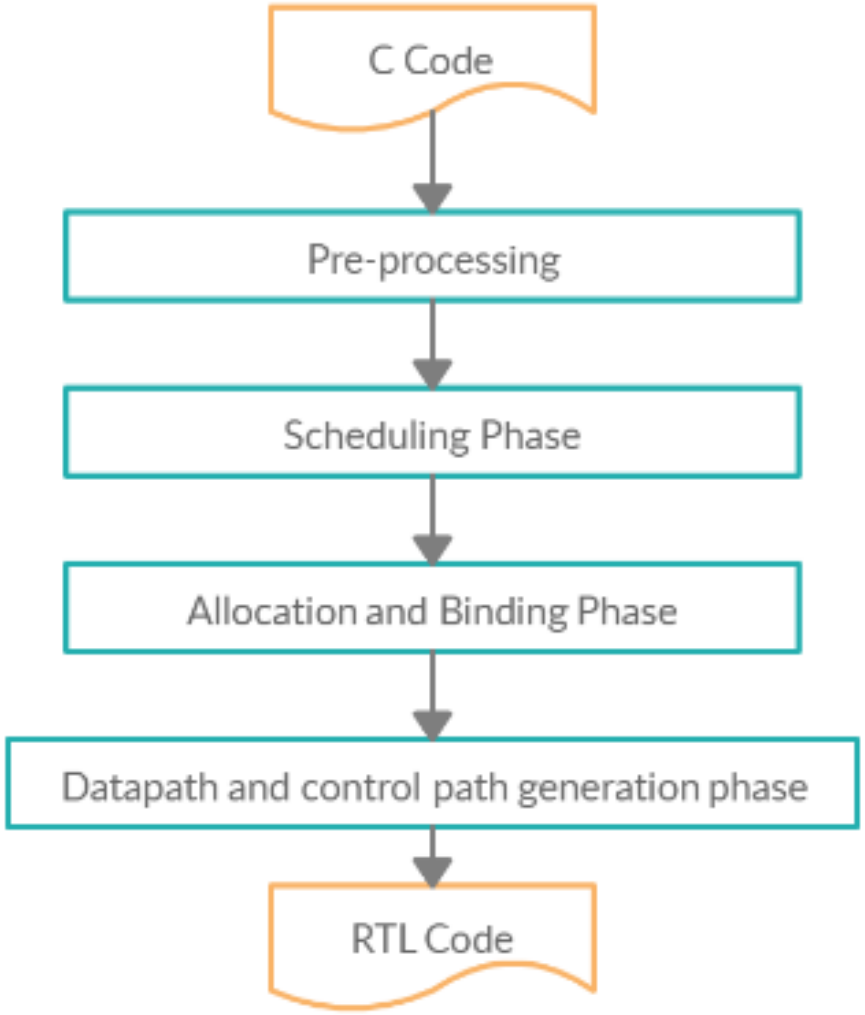
Dr. Chandan Karfa

Department of Computer Science and Engineering

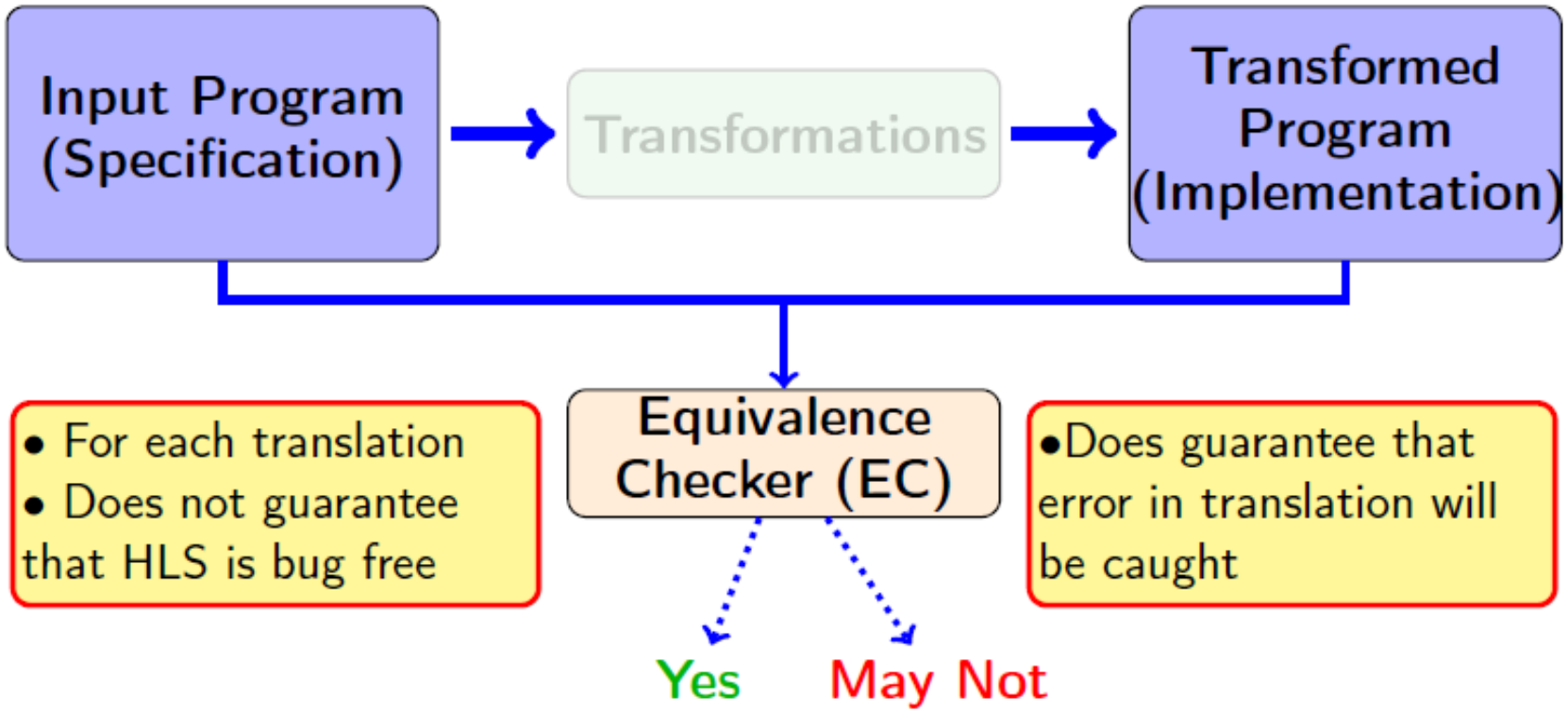


भारतीय प्रौद्योगिकी संस्थान गुवाहाटी
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI

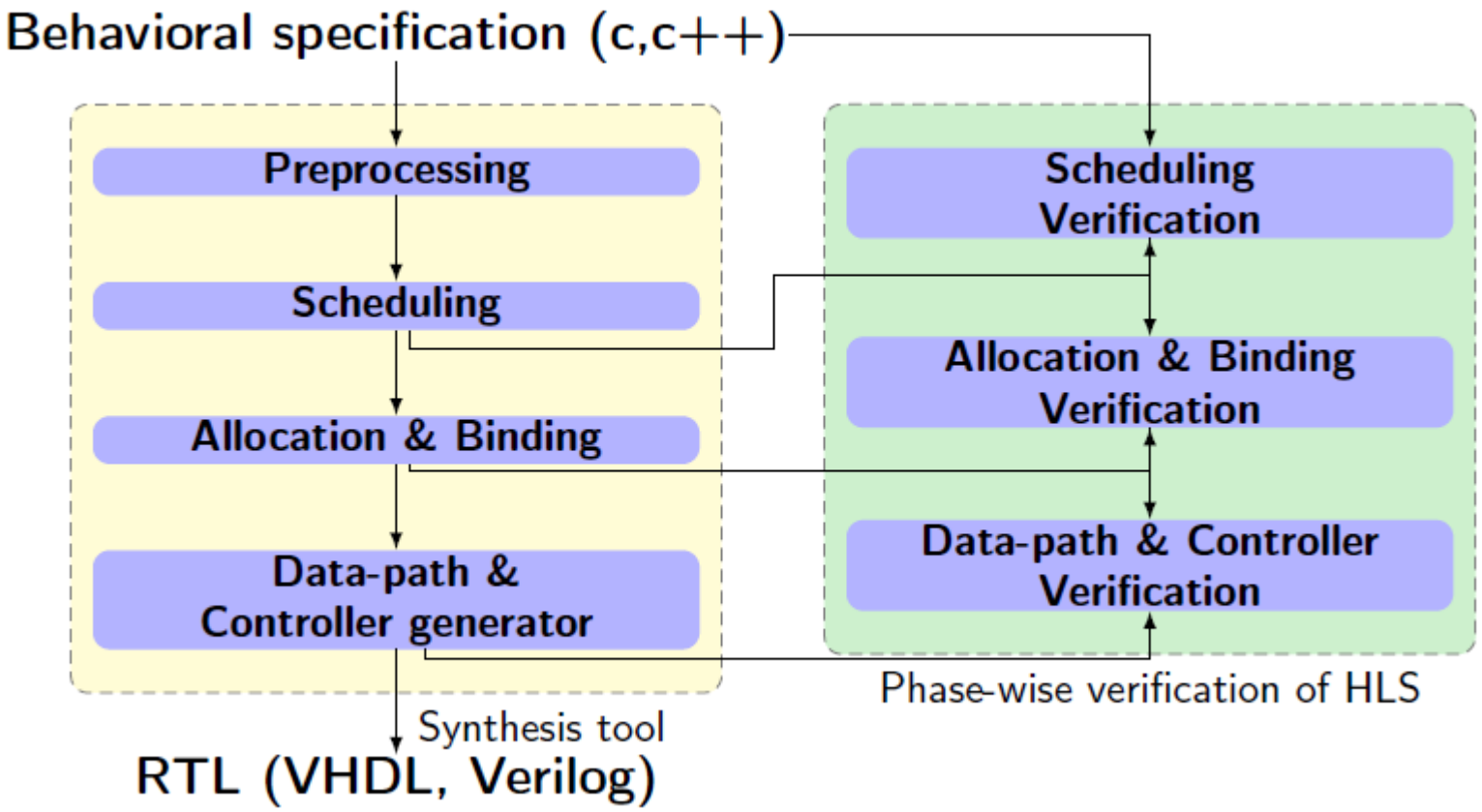
HLS Flow


C Code
Pre-processing
Scheduling Phase
Allocation and Binding Phase
Datapath and control path generation phase
RTL Code

Translation Validation of HLS



Phase-wise Verification of HLS

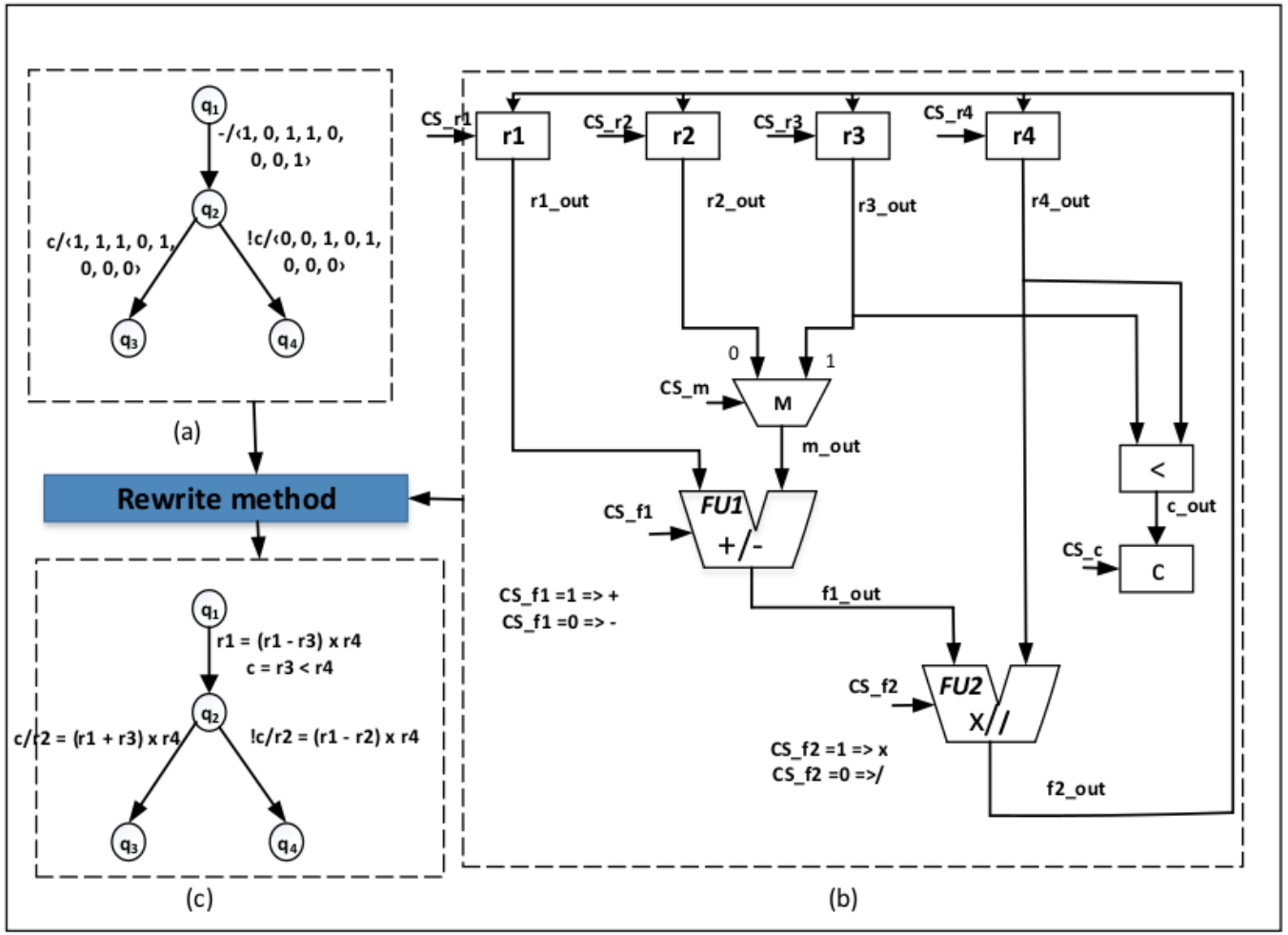


- End-to-end verification is not only tough but also inadequate to locate the exact source of error.
- Ability to verify each sub-task more carefully.
- Ability to use different verification technique for different phase

C to RTL Equivalence Checking

- Need intermediate information from the HLS tool.
  - May not be available
- End to end verification does not need any intermediate information from the tool.
  - Difficult problem due to semantic gap between C and Verilog/Vhdl.
  - No commercial solution exists
- Need some sort of similar representation.
  - RTL to C conversion is helpful here

RTL to C Conversion




```
Void main(){
  int r1, r2, r3, r4;
  q1: //state label
       r1 = (r1 – r3) x r4;
      c = r3 < r4;
      go to q2;
  q2:
     if(c){
          r2 = (r1 + r3) x r4;
          go to q3;}
     else{
          r2 = (r1 – r3) x r4;
          go to q4;}
  q3:
    …….
  q4:
    ……..
```

How C and RTL-C look like?

- Inputs and outputs are the same
- Control structures are different
- The internal variables are completely different.
- Why the path based approaches commonly used for scheduling verification cannot be applied here?
  - Computation/trace level equivalence is the feasible way unless the co-relation among the variables among two programs are identified.
  - Therefore loop must be of static bound
- Is static loop bound too restrictive in HLS?
  - NO

Program Model

- Finite State machine with Data paths
- Path
  - Condition of execution and data transformations of a path
- Computation/trace
  - Condition of execution and data transformations of a path

Equivalence of two FSMDs :

For any execution trace in one FSMD, there is an equivalent one in the other FSMD. For any computation/trace c_0 of $M_{0,}$ there exists a computation/trace c_1 in M_1 so that $c_0 \approx c_1$ and vice-versa.

- Two traces are equivalent if condition of executions are equivalent and the respective outputs are equivalent.
- Condition executions and the output expressions are symbolic expressions over Inputs and constants.

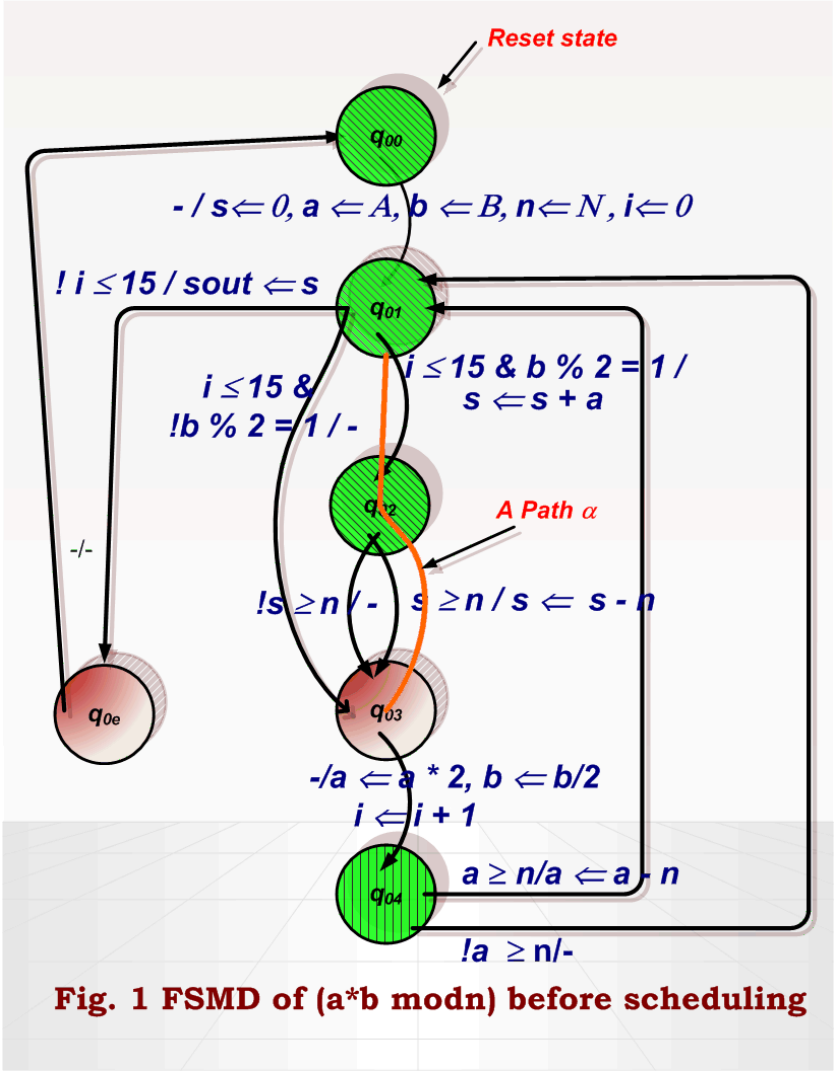
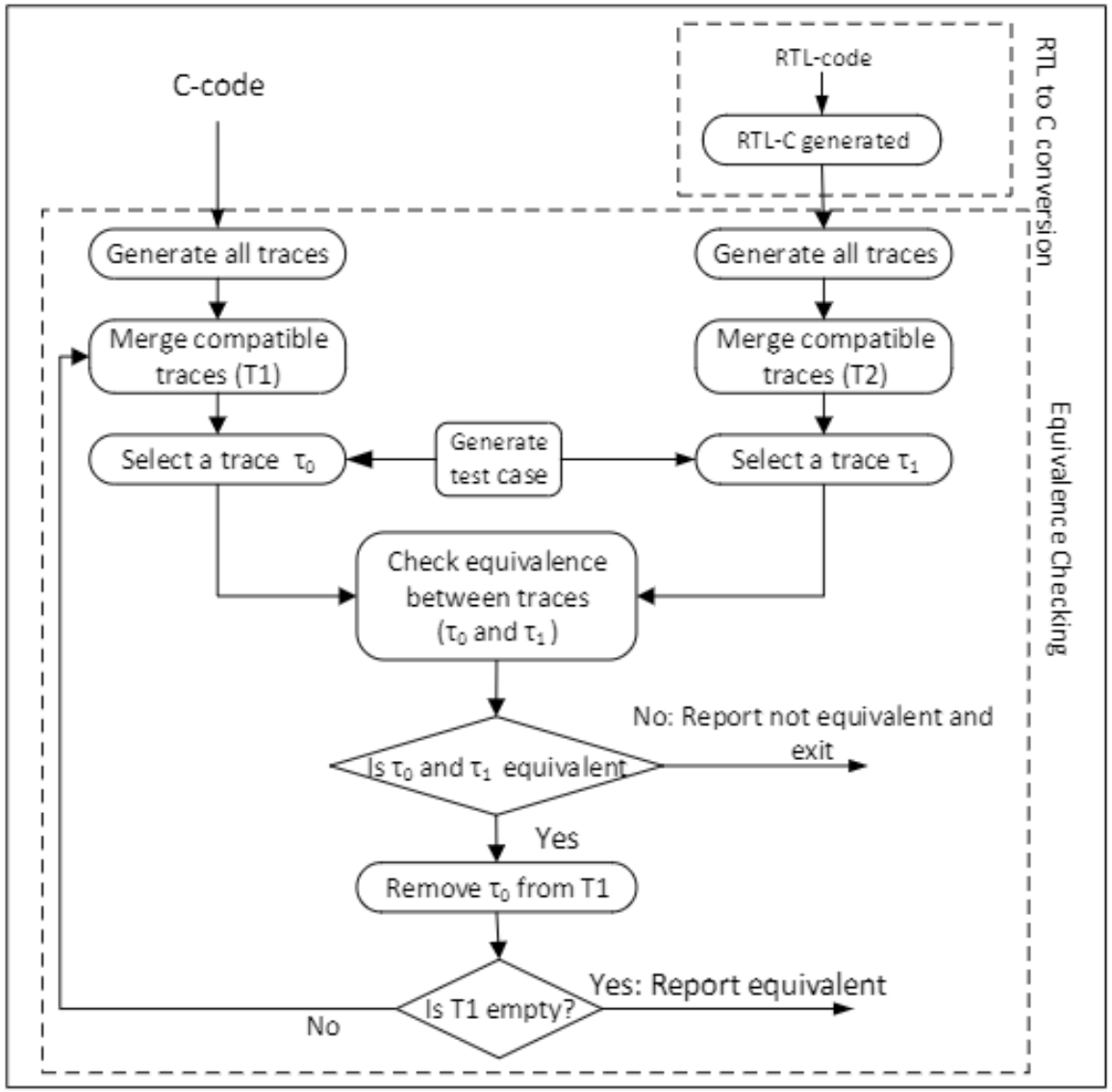


Fig. 1 FSMD of (a*b modn) before scheduling

C to RTL Equivalence Checking Flow



Generate all traces in both the behaviours

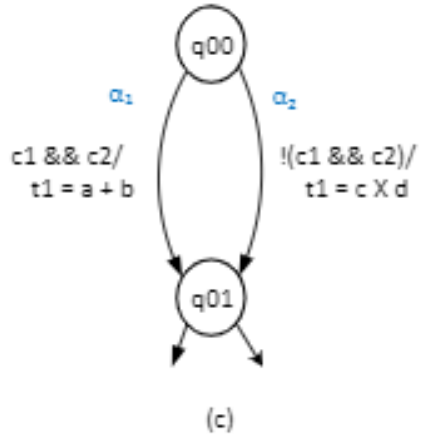
- We have Used KLEE to identify all traces and their symbolic condition of executions and output expressions

```
if (c1 && c2){
   t1 = a + b;
}
else{
  t1 = c x d;
}
```

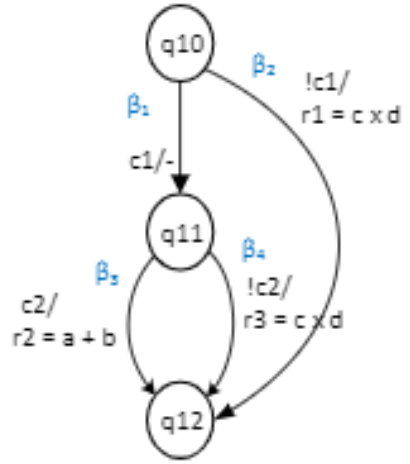
(a)

```
if(c1){
    if(c2)
        t1 = a + b;
    else
        t1 = c x d;
}
else{
    t1 = c x d;
}
```

(b)



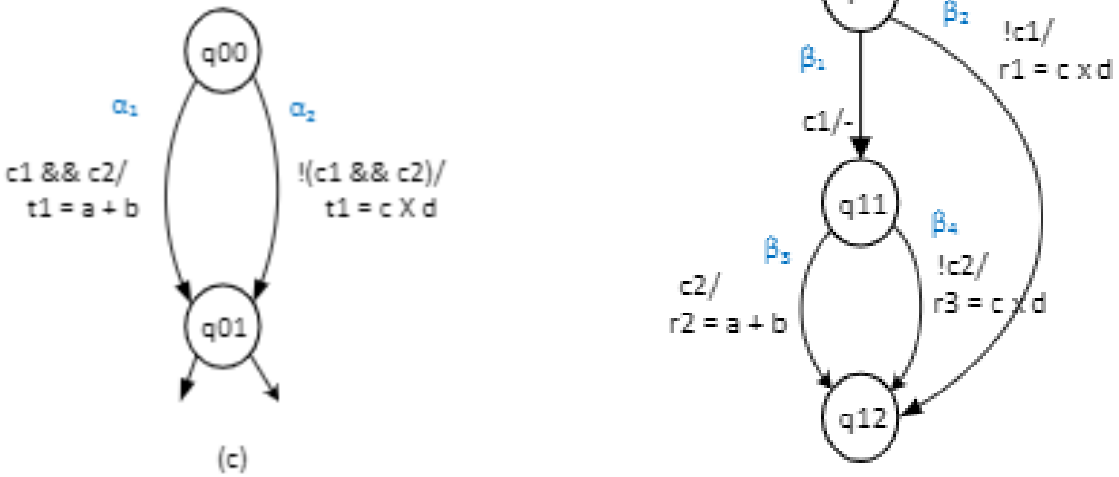
(c)



https://klee.github.io/

Merge compatible traces

- To improve performance of equivalence checking, traces which have same output expression, i.e., compatible traces, in each behaviour are merged.



- Number of traces may not be the same even after merging compatible traces

Find potential corresponding traces between two behaviours

- For each trace in M1, we need to find an equivalent trace in M2.
  - Complexity: $O(n^2)$, where n is the number of traces
- Can we reduce the complexity?
  - Use data-driven approach.
  - Use KLEE for the same

Checking one to one equivalence

- For each potential corresponding traces, we check if their respective condition of executions and data transformations are equivalent
- Use SMT Solver for formally prove the equivalence of potential corresponding traces.

Checking one to many equivalence

- In some cases one to one equivalence can not be obtain for a subset of traces.
- For each of such trace, we identify iteratively a set of traces that are equivalent to a trace

Algorithm 1: C_to_RTL_EqCheck (C, RTL-C)

Input: Input-C, RTL-C
Result: Equivalent, May not equivalent

```
1  T0 = findTrace(C); T1 = findTrace(RTL-C);
2  T0 = mergeTrace (T0); T1 = mergeTrace (T1);
3  copyT0= T0; copyT1= T1; flag = 0;
4  while T0 ≠ φ do
5      τ0 = select a trace from T0;
6      TC = getTestcase(τ0);
7      τ1 = getCorrespondingTrace(T1, TC);
8      if ((c_τ0 ≡ c_τ1) ∧ (s_τ0 ≡ s_τ1)) then
9          //τ0 and τ1 are equivalent
10         removeTrace (τ0, copyT0);
11         removeTrace (τ1, copyT1);
12     else if ((c_τ0 ≡ c_τ1) ∧ (s_τ0 ≠ s_τ1)) then
13         //τ0 and τ1 may not equivalent
14         Report "May not equivalent (MNE)" and Exit;
15     else
16         flag=1;
17     endif
18     removeTrace (τ0, T0);
19 endwhile
20 if (flag = 1) then
21     foreach τ0 ∈ copyT0 do
22         foreach τ1 ∈ copyT1 do
23             if (c_τ0 ∧ c_τ1 ≠ φ) then
24                 if (c_τ0 ∧ c_τ1 ∧ s_τ0 ≠ s_τ1) then
25                     Report "May not equivalent" and Exit;
26 Report Equivalent (Eq);
```

Experimental Results

Bench	#in	#out	C code		RTL code		RTL-C		Traces			Equivalent		Not Equivalent	
			#line	#var	#line	#regs	#line	#var	#C	#RTLC	#merged	time (s)	result	time (s)	result
(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)	(13)	(14)	(15)	(16)
Waka	20	3	33	32	270	12	382	126	3	4	(3, 3)	1.709	Eq	0.669s	MNE
Arf	11	4	53	43	351	19	607	158	4	4	(3, 3)	1.890	Eq	0.949	MNE
Parker	6	1	62	14	196	10	275	100	12	23	(2, 2)	1.614	Eq	0.976	MNE
FindMin8	8	1	40	15	175	11	780	243	128	128	(8, 8)	22.246	Eq	17.141	MNE
MatrixAdd	2	1	48	7	734	44	2595	241	1	1	(1, 1)	1.684	Eq	0.749	MNE
SumArray	1	1	19	4	263	15	541	100	1	1	(1, 1)	0.754	Eq	0.706	MNE
Motion	10	3	52	43	413	29	881	235	1	1	(1, 1)	0.681	Eq	0.663	MNE
Dfadd	2	1	719	70	1975	113	9353	1041	67	68	(21,42)	1016.052	Eq	960.238	MNE

Summary

- We have proposed an RTL to C translation validation framework for verification HLS results.
- The innovative part of our framework to improve the efficiency
  - Extraction of a RTL-C from the RTL to reduce the semantic gap between C and RTL
  - Merging compatible traces within a behaviour to handle control structure modification during HLS
  - Use of a data-driven approach to find the potential equivalent trace pairs between two behaviours.
  - The equivalence of potential equivalence traces are then formally proved using an SMT solver.

Thank You